

Verification and Validation

- Assuring that a software system conforms to its specification and meets the user's needs
- Verification:
"Are we building the product right?"
i.e., the software should conform to its specification
- Validation:
"Are we building the right product?"
i.e., the software should do what the user really requires

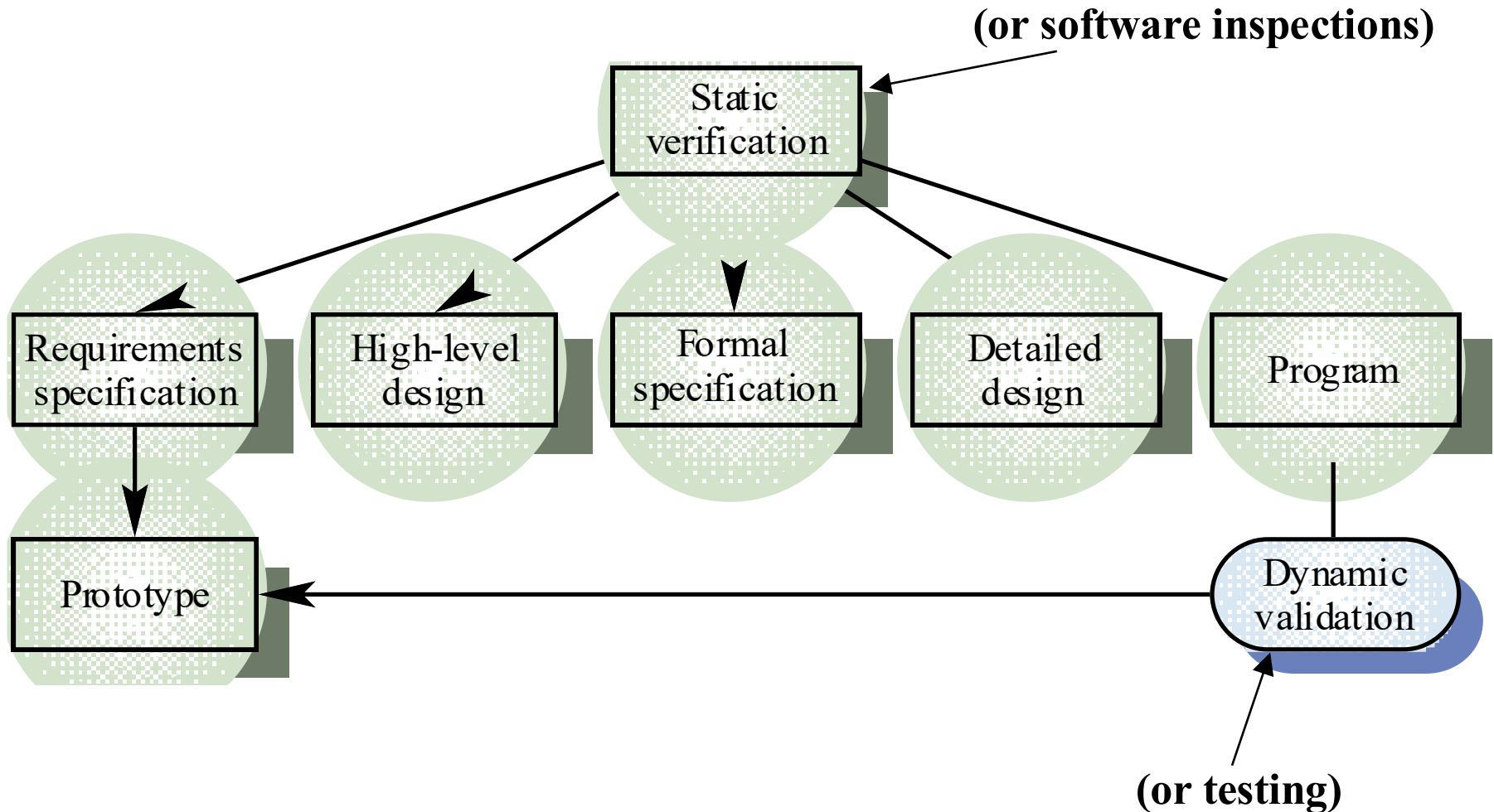
The V&V process

- Is a whole life-cycle process - V&V must be applied at each stage in the software process.
- Has two principal objectives
 - The discovery of defects in a system
 - The assessment of whether or not the system is usable in an operational situation
- V&V should be *independent*, which means that the development team and the SQA team should be separate teams (different managers and different members)

V&V techniques

- *Software inspections* Concerned with analysis of the static system representation to discover problems (*static techniques*)
 - May be supplemented by tool-based document and code analysis
- *Software testing* Concerned with exercising and observing product behaviour (*dynamic techniques*)
 - The system is executed with test data and its operational behaviour is observed

Static and dynamic V&V



Dynamic validation (testing)

- ***Can reveal the presence of defects NOT their absence***
- A successful test is a test which discovers one or more defects
- The only validation technique for non-functional user requirements
 - Dynamic validation of non-executable artifacts can be carried out using simulation-based approaches
- Should be used in conjunction with static verification to provide full V&V coverage
 - Static techniques can only check the correspondence between a program and its specification (verification); they cannot demonstrate that the software is operationally useful (validation)

Types of testing

- Validation testing
 - Tests intended to demonstrate that a system meets its user requirements
 - A successful validation test requires the system to perform correctly using given acceptance test cases
- Defect testing
 - Tests designed to discover system defects
 - A successful defect test is one which reveals the presence of defects in a system
- Statistical testing
 - Tests designed to reflect the frequency of user inputs
 - Used for *reliability* estimation

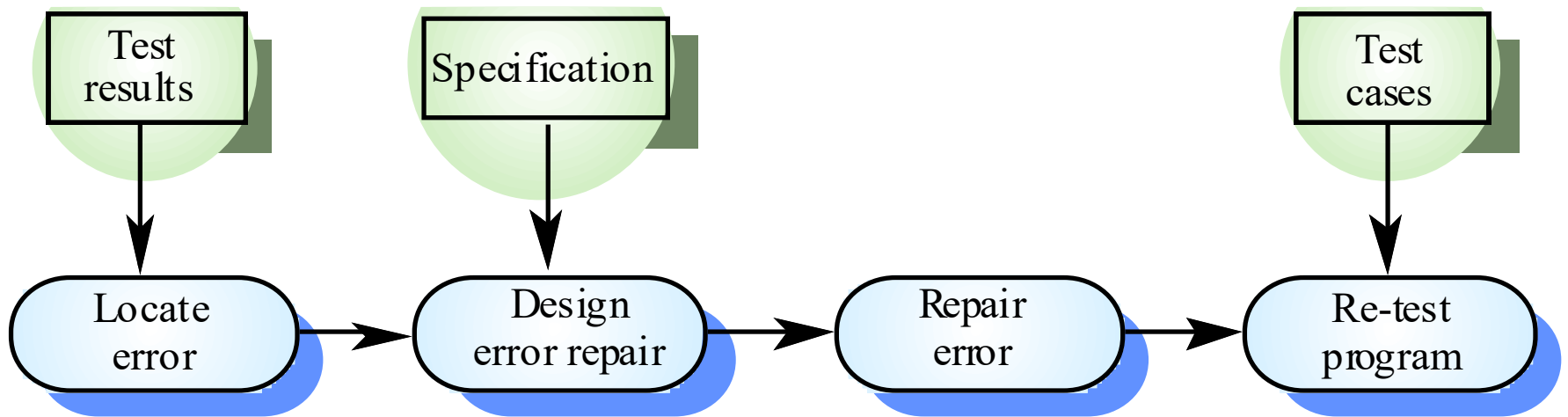
V&V goals

- V&V should establish confidence that the software is *fit for purpose*
- This does NOT mean that the system must be completely free of defects, but good enough for its intended use and the type of use will determine the degree of confidence that is needed, which depends on:
 - System's purpose
 - The level of confidence depends on how critical the software is to an organisation
 - User expectations
 - Users may have low expectations of certain kinds of software
 - Marketing environment
 - Getting a product to market early may be more important than finding defects in the program

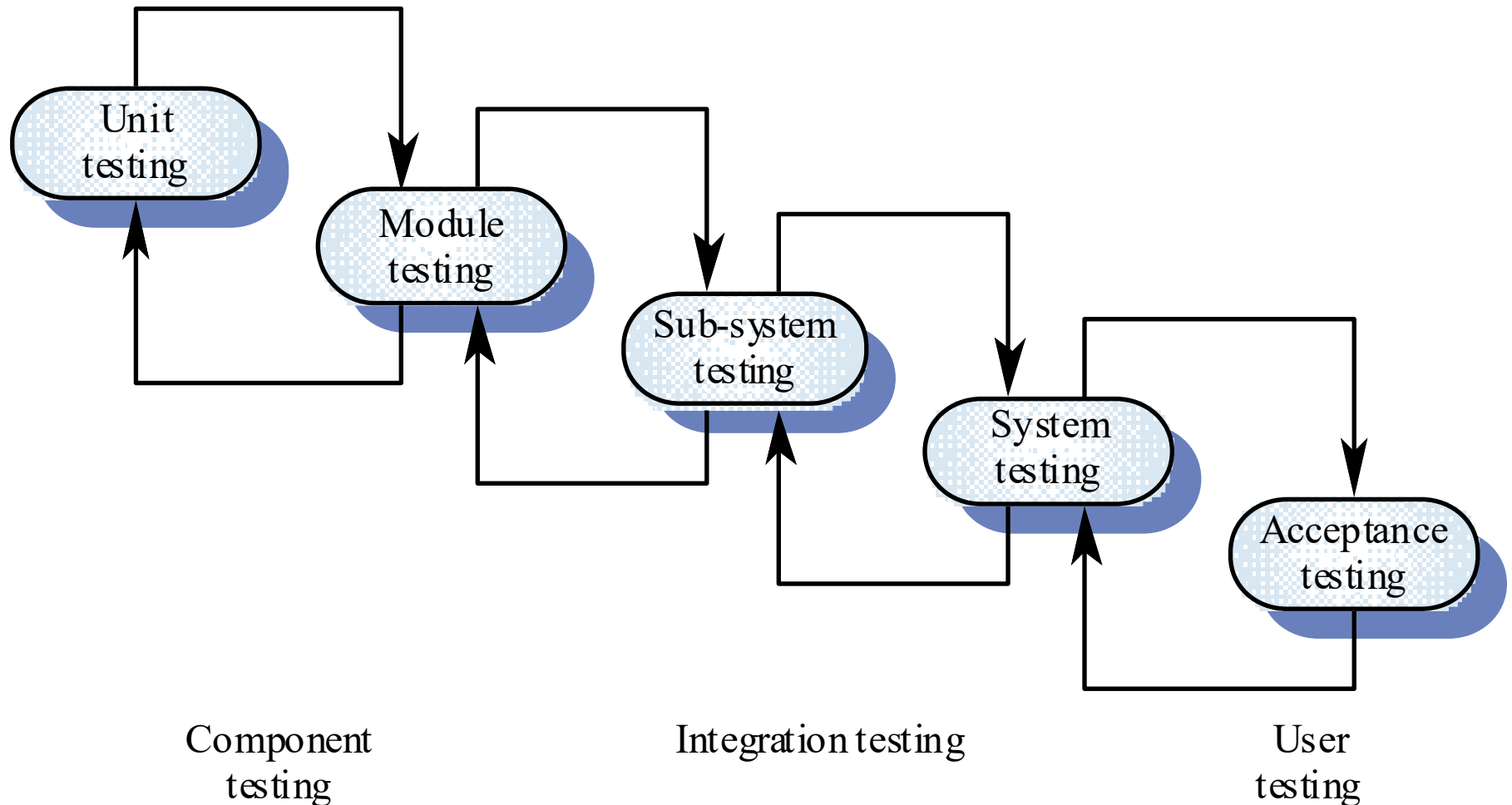
Testing vs. debugging

- Defect testing and debugging are distinct processes
- Verification and validation is concerned with establishing the existence of defects in a program
- Debugging is concerned with locating and repairing these defects
- Debugging involves formulating a hypothesis about program behaviour then testing these hypotheses to find the system defect

The debugging process



The testing process



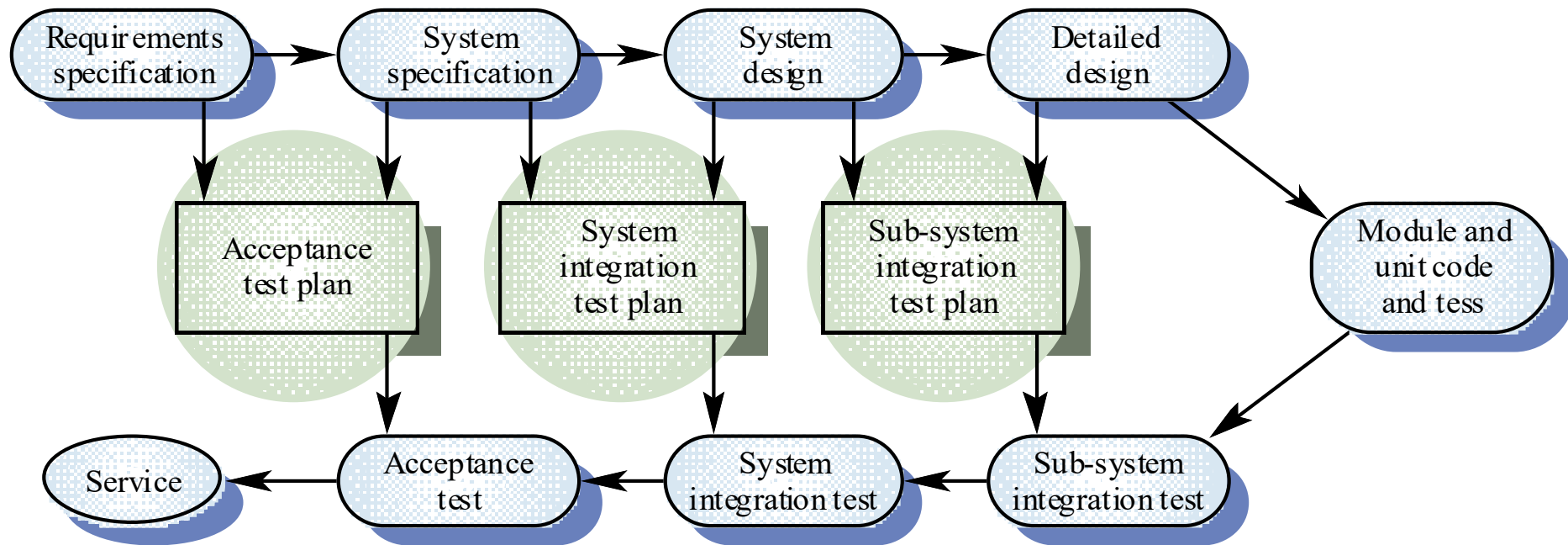
Testing stages

- Unit testing
 - Individual components are tested
- Module testing
 - Related collections of dependent components are tested
- Sub-system testing
 - Modules are integrated into sub-systems and tested. The focus here should be on interface testing
- System testing
 - Testing of the system as a whole. Testing of emergent properties
- Acceptance testing
 - Testing with customer data to check that it is acceptable

V&V planning

- Careful planning is required to get the most out of testing and inspection processes
- Planning should start early in the development process
- The plan should identify the balance between static verification and testing
- Test planning is about defining standards for the testing process rather than describing product tests

The V-model of development



The structure of a software test plan

- The testing process
- Requirements traceability
- Tested items
- Testing schedule
- Test recording procedures
- Hardware and software requirements
- Constraints

Software inspections

- Involve people examining the source representation with the aim of discovering anomalies and defects
- Do not require execution of a system so may be used before implementation
- May be applied to any representation of the system (requirements, design, test data, etc.)
- Very effective technique for discovering defects:
 - *Fagan* (1986) reported that more than 60% of the defects in a program can be detected using software inspections
 - *Mills et al.* (1987) suggest that a more formal approach, using mathematical verification, can detect more than 90% of the defects in a program

Inspections and testing

- Inspections and testing are complementary and not opposing verification techniques
- Both should be used during the V&V process
- Inspections can check conformance with a specification but not conformance with the customer's real requirements
- Inspections cannot check non-functional characteristics such as performance, usability, etc.

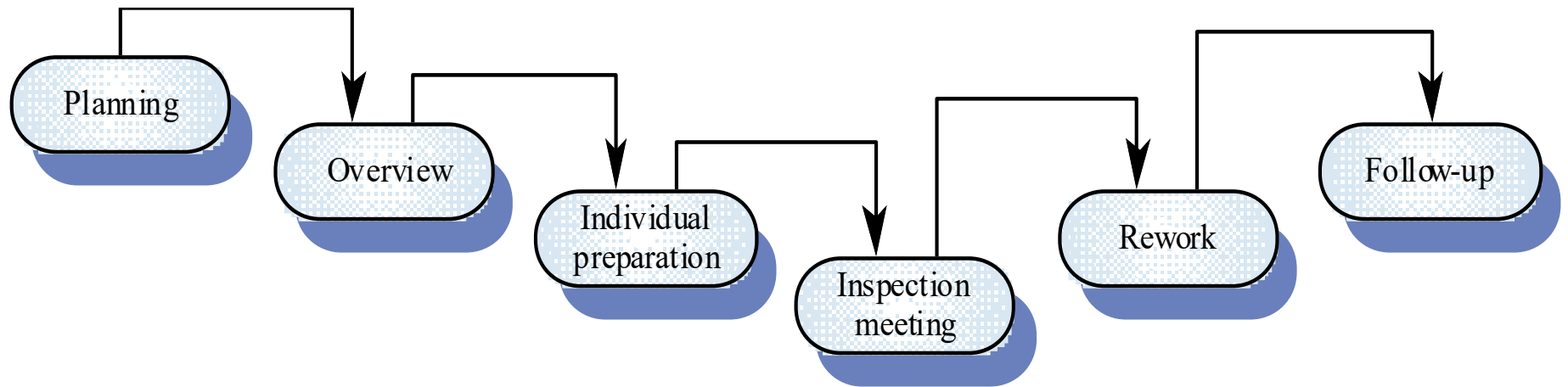
Program inspections

- Formalised approach to document reviews
- Intended explicitly for defect DETECTION (not correction)
- Defects may be logical errors, anomalies in the code that might indicate an erroneous condition (e.g. a not-initialised variable) or non-compliance with standards

Inspection pre-conditions

- A precise specification must be available
- Team members must be familiar with the organisation standards
- Syntactically correct code must be available
- A defect checklist should be prepared
- Management must accept that inspection will increase costs early in the software process
- Management must not use inspections for staff appraisal

The inspection process



Inspection procedure

- System overview presented to inspection team
- Code and associated documents are distributed to inspection team in advance
- Inspection takes place and discovered defects are noted
- Modifications are made to repair discovered defects
- Re-inspection may or may not be required

Inspection teams

- Made up of at least 4 members
 - **Author:** of the code being inspected
 - **Inspector:** who finds defects, omissions and inconsistencies
 - **Reader:** who reads the code to the team
 - **Moderator:** who chairs the meeting and notes discovered defects
 - Other roles are Scribe and Chief moderator

Inspection checklists

- Checklist of common defects should be used to drive the inspection
- Defects checklist is programming language dependent
- The 'weaker' the type checking, the larger the checklist
- Examples: Initialisation, constant naming, loop termination, array bounds, etc.

Fault class	Inspection check
Data faults	Are all program variables initialised before their values are used? Have all constants been named? Should the lower bound of arrays be 0, 1, or something else? Should the upper bound of arrays be equal to the size of the array or Size -1? If character strings are used, is a delimiter explicitly assigned?
Control faults	For each conditional statement, is the condition correct? Is each loop certain to terminate? Are compound statements correctly bracketed? In case statements, are all possible cases accounted for?
Input/output faults	Are all input variables used? Are all output variables assigned a value before they are output?
Interface faults	Do all function and procedure calls have the correct number of parameters? Do formal and actual parameter types match? Are the parameters in the right order? If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	If a linked structure is modified, have all links been correctly reassigned? If dynamic storage is used, has space been allocated correctly? Is space explicitly de-allocated after it is no longer required?
Exception management faults	Have all possible error conditions been taken into account?

Inspection checks

Inspection rate

- 500 statements/hour during overview
- 125 source statements/hour during individual preparation
- 100-125 statements/hour can be inspected
- Inspection is therefore an expensive process
- Inspecting 500 statements costs about 40 man/hours of effort

Automated static analysis

- Static analysers are software tools for source text processing
- They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the V&V team
- Very effective as an aid to inspections. A supplement to but not a replacement for inspections

Static analysis checks

Fault class	Static analysis check
Data faults	Variables used before initialisation Variables dedared but never used Variables assigned twice but never used between assignments Possible array bound violations Undeclared variables
Control faults	Unreachable code Unconditional branches into loops
Input/output faults	Variables output twice with no intervening assignment
Interface faults	Parameter type mismatches Parameter number mismatches Non-usage of the results of functions Uncalled functions and procedures
Storage management faults	Unassigned pointers Pointer arithmetic

Stages of static analysis

- *Control flow analysis.* Checks for loops with multiple exit or entry points, finds unreachable code, etc.
- *Data use analysis.* Detects not-initialised variables, variables assigned twice without an intervening assignment, variables which are declared but never used, etc.
- *Interface analysis.* Checks the consistency of routine and procedure declarations and their use

Stages of static analysis

- *Information flow analysis.* Identifies the dependencies of output variables. Does not detect anomalies itself but highlights information for code inspection or review
- *Path analysis.* Identifies paths through the program and sets out the statements executed in that path. Again, potentially useful in the review process
- Both these stages generate vast amounts of information. Must be used with care.

```
>> more lint_ex.c
```

```
#include <stdio.h>
printarray (Anarray)
    int Anarray;
{
    printf("%d",Anarray);
}
main ()
{
    int Anarray[5]; int i; char c;
    printarray (Anarray, i, c);
    printarray (Anarray) ;
}
```

```
>> cc lint_ex.c
```

```
>> lint lint_ex.c
```

```
lint_ex.c(10): warning: c may be used before set
lint_ex.c(10): warning: i may be used before set
printarray: variable # of args. lint_ex.c(4) :: lint_ex.c(10)
printarray, arg. 1 used inconsistently lint_ex.c(4) :: lint_ex.c(10)
printarray, arg. 1 used inconsistently lint_ex.c(4) :: lint_ex.c(11)
printf returns value which is always ignored
```

LINT

static analysis

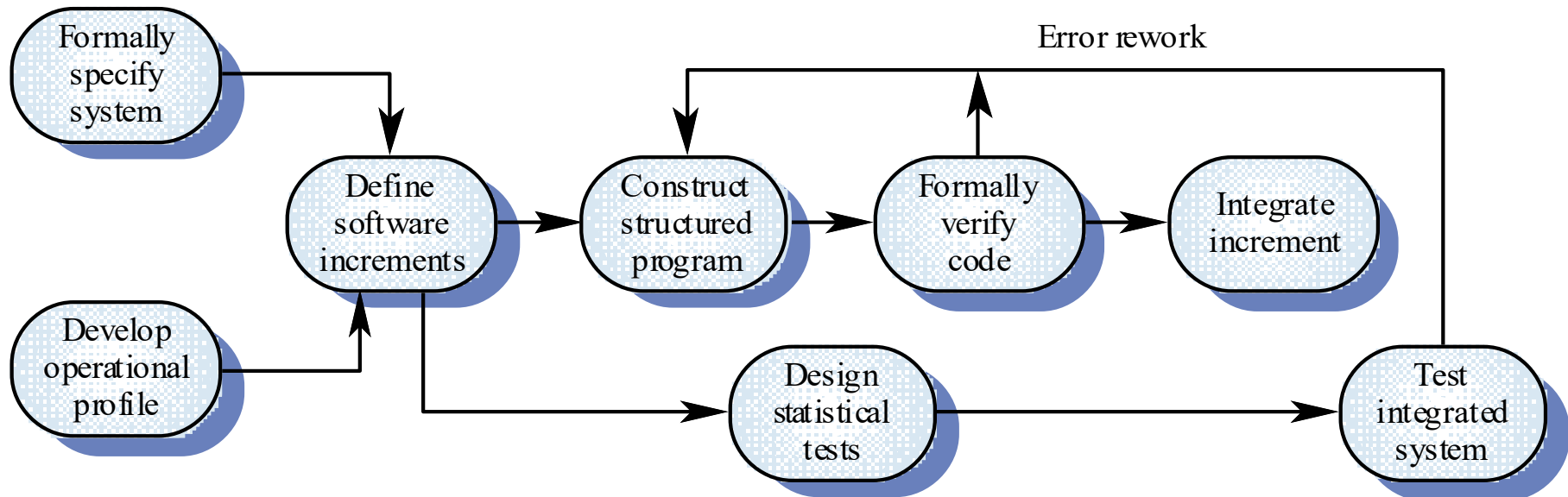
Use of static analysis

- Particularly valuable when a language such as C is used which has weak typing and hence many defects are undetected by the compiler
- Less cost-effective for languages like Java that have strong type checking and can therefore detect many defects during compilation

Cleanroom software development

- The name is derived from the 'Cleanroom' process in semiconductor fabrication. The philosophy is *defect avoidance* rather than defect removal
- Software development process based on:
 - Incremental development
 - Formal specification
 - Static verification using correctness arguments
 - Statistical testing to determine program reliability

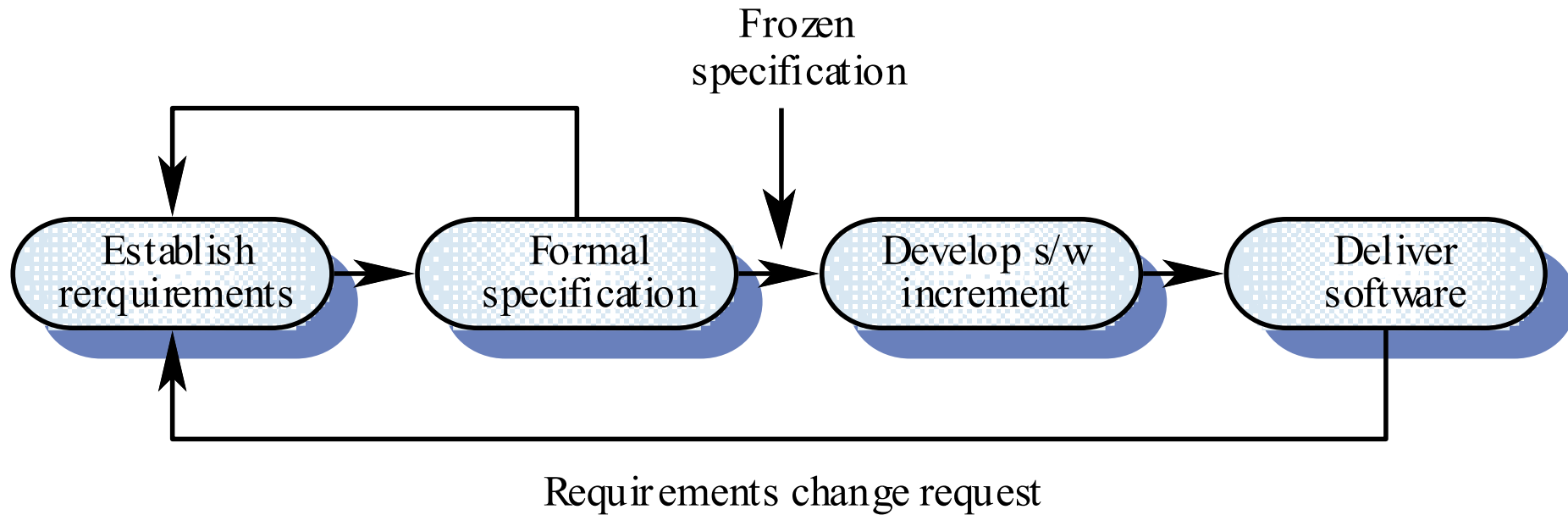
The Cleanroom process



Cleanroom process characteristics

- Formal specification using a state transition model
- Incremental development
- Structured programming - limited control and abstraction constructs are used
- Static verification using rigorous inspections
- Statistical testing of the system

Incremental development



Formal specification and inspections

- The state based model is a system specification and the inspection process checks the program against this model
- Programming approach is defined so that the correspondence (transformation) between the model and the system is clear
- Mathematical arguments (not proofs) are used to increase confidence in the inspection process

Cleanroom process teams

- *Specification team.* Responsible for developing and maintaining the system specification
- *Development team.* Responsible for developing and verifying the software. The software is NOT executed or even compiled during this process
- *Certification team.* Responsible for developing a set of statistical tests to exercise the software after development. Reliability growth models are used to determine when to stop testing (i.e., when reliability is acceptable)

Cleanroom process evaluation

- Results in IBM have been very impressive with few discovered faults in delivered systems
- Independent assessment shows that the process is no more expensive than other approaches
- Fewer defects than in a 'traditional' development process
- Not clear how this approach can be transferred to an environment with less skilled or less highly motivated engineers

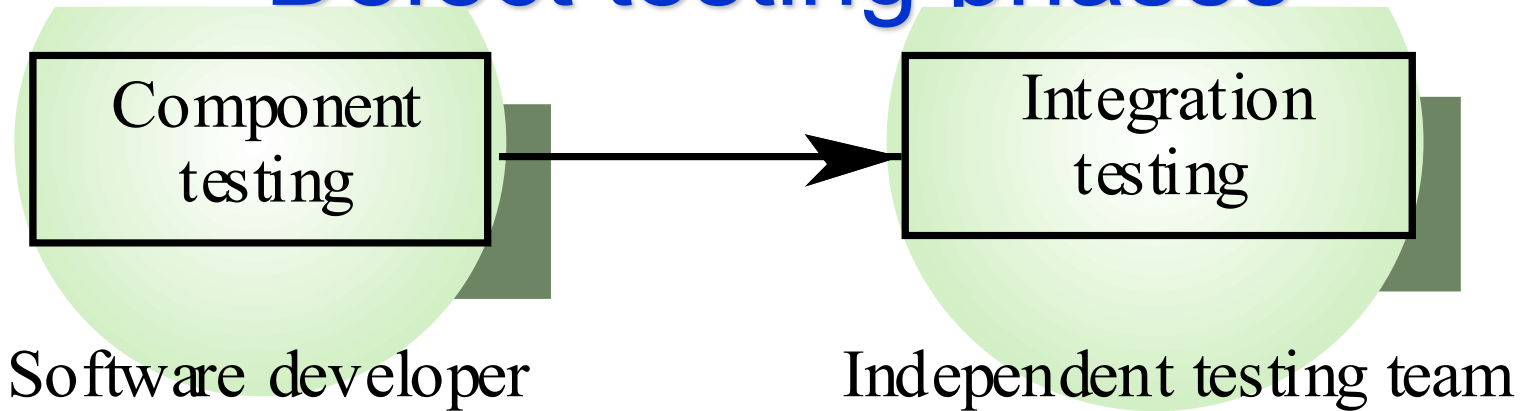
Types of testing

- Validation testing
 - Tests intended to demonstrate that a system meets its user requirements
 - A successful validation test requires the system to perform correctly using given acceptance test cases
- Defect testing 
 - Tests designed to discover system defects
 - A successful defect test is one which reveals the presence of defects in a system
- Statistical testing
 - Tests designed to reflect the frequency of user inputs
 - Used for *reliability* estimation

Defect testing

- The goal of defect testing is to discover defects in programs
- This contrast with *validation testing* which is intended to demonstrate that a system meets its user requirements
- Validation testing requires the system to perform correctly using given acceptance test cases
- A *successful* defect test is a test which causes a program to behave in an anomalous way
- Tests show the presence not the absence of defects

Defect testing phases



- **Component testing** (*unit* and *module* testing)
 - Testing of individual program components
 - Usually the responsibility is of the component developer (except sometimes for critical systems)
 - Tests are derived from the developer's experience
- **Integration testing** (*sub-system* and *system* testing)
 - Testing of groups of components integrated to create a system or sub-system
 - The responsibility is of an independent testing team
 - Tests are based on a system specification
- **User testing** (*validation* or *acceptance* testing) is not part of the defect testing process

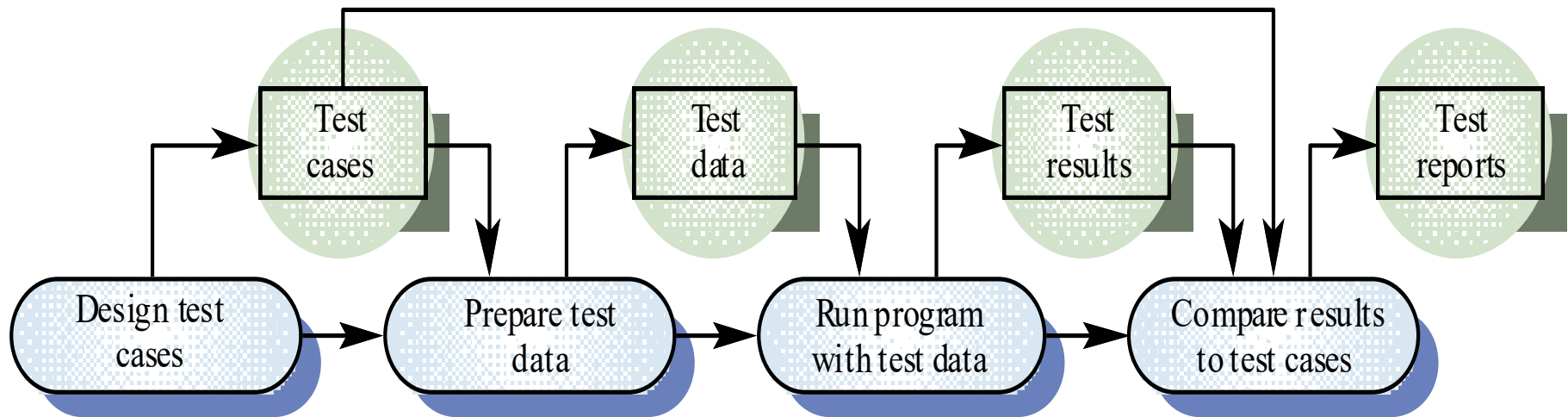
Testing policies

- Only exhaustive testing can show that a program is free from defects. However, exhaustive testing is impossible
- Testing must be based on a subset of possible test cases, according to policies which should be devised by the V&V team (and not by the development team)
- Tests should exercise a system's capabilities rather than its components
- Testing typical situations is more important than boundary value cases

Test cases and test data

- *Test cases*
 - Inputs to test the system and the predicted outputs from these inputs if the system operates according to its specification
 - Test cases are usually generated manually (because it is not easy to automatically derive test output from informal specifications)
- *Test data*
 - Inputs which have been devised to test the system
 - Test data can be generated automatically

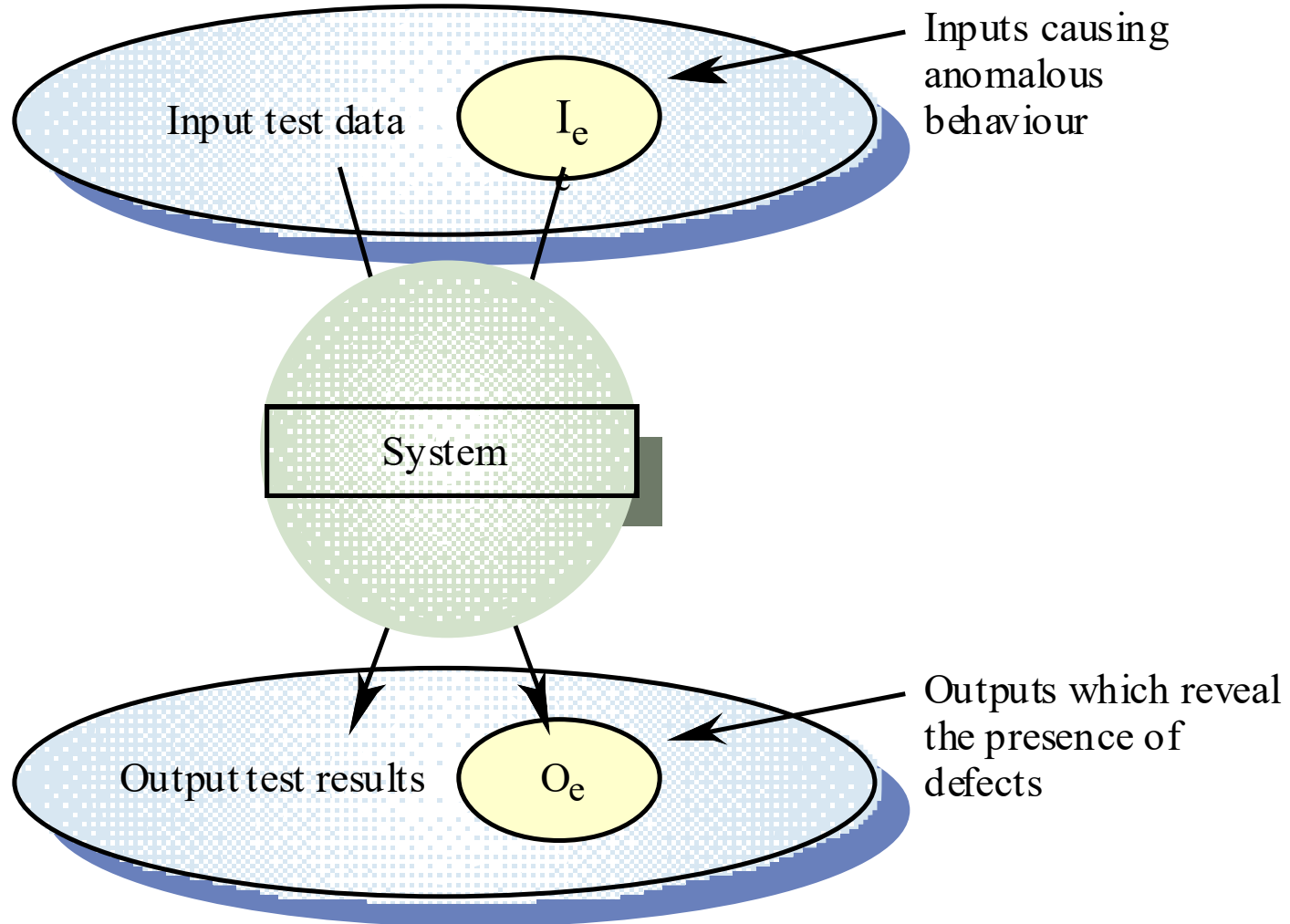
The defect testing process



Black-box testing

- An approach to testing where the program is considered as a *black-box*
- The program test cases are derived from the *system specification*
- The tester presents inputs to the component or system and examines the corresponding output
- If the outputs are not those specified then the test has *successfully* detected a problem with the software
- Also called *functional testing* because the tester is only concerned with the functionality and not the implementation of the software

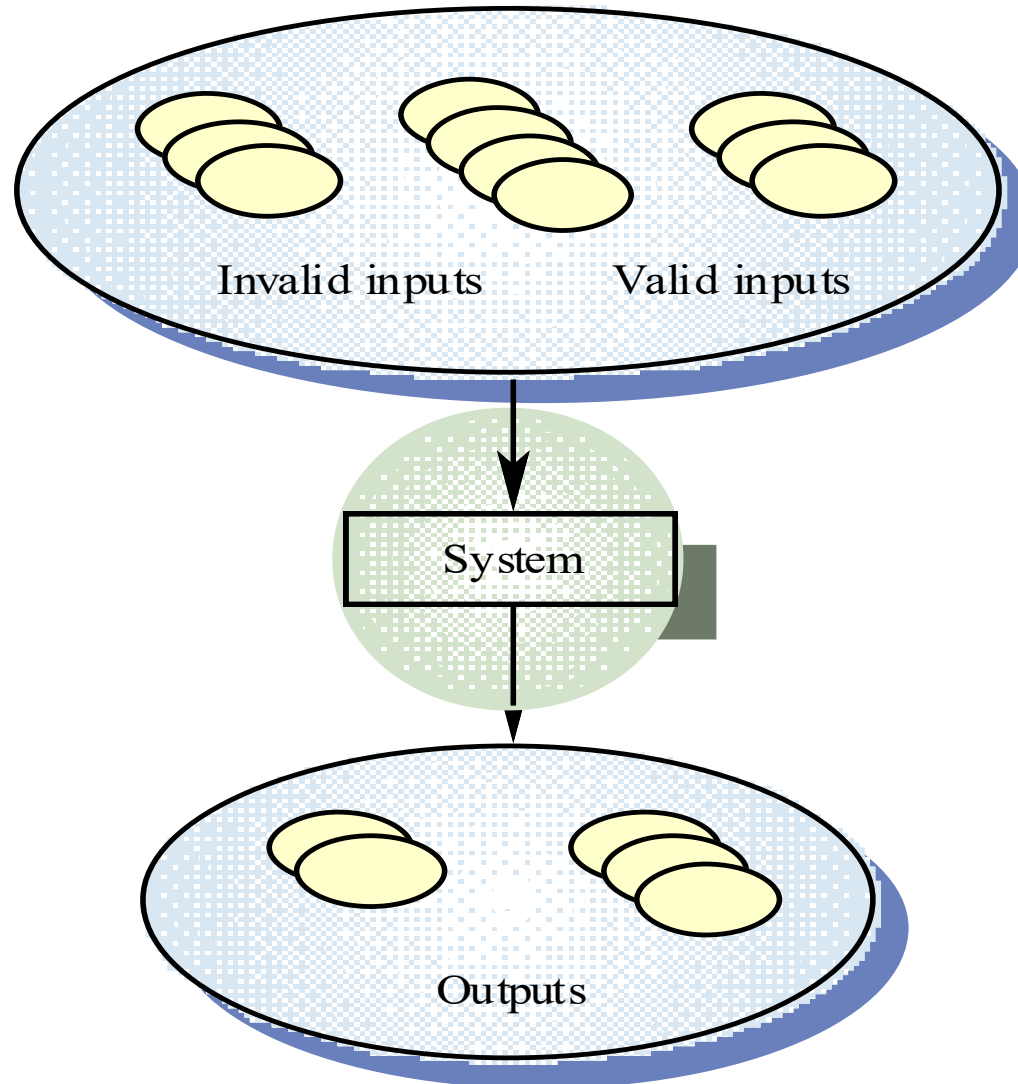
Black-box testing



Equivalence partitioning

- Input data and output results often fall into different classes where all members of a class are related
- Each of these classes is an *equivalence partition* where the program behaves in an equivalent way for each class member
- Test cases should be chosen from each partition

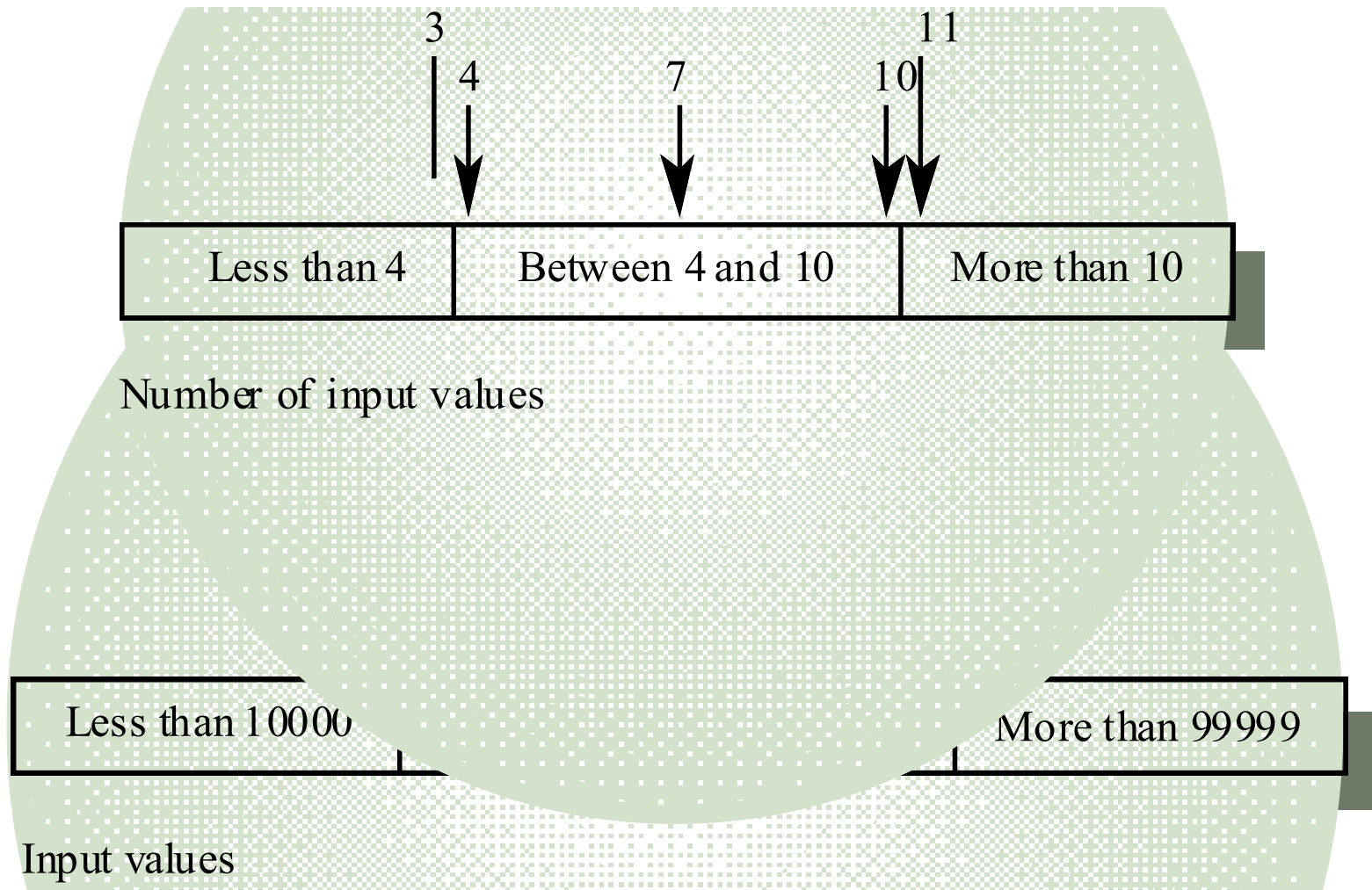
Equivalence partitioning



Equivalence partitioning

- Partition system inputs and outputs into 'equivalence partitions'
 - If input is a 5-digit integer between 10,000 and 99,999, equivalence partitions are
 $< 10,000 \mid 10,000-99,999 \mid > 99,999$
- Choose test cases at the boundary of these partitions plus cases close to the mid-point of the partitions with valid inputs
 - 09999, 10000, 50000, 99999, 100000

Equivalence partitions



Search routine specification

procedure Search (Key : ELEM ; T: ELEM_ARRAY;
Found : **in out** BOOLEAN; L: **in out** ELEM_INDEX) ;

Pre-condition

-- the array has at least one element
T'FIRST <= T'LAST

Post-condition

-- the element is found and is referenced by L
(Found and T (L) = Key)

or

-- the element is not in the array
(**not** Found **and**
not (**exists** i, T'FIRST >= i <= T'LAST, T (i) = Key))

Search routine - input partitions

- Inputs which conform to the pre-conditions
- Inputs where a pre-condition does not hold
- Inputs where the key element is a member of the array
- Inputs where the key element is not a member of the array

Testing guidelines (sequences)

- Test software with sequences which have only a single value
- Use sequences of different sizes in different tests
- Derive tests so that the first, middle and last elements of the sequence are accessed
- Test with sequences of zero length

Search routine

Example input partitions and test cases

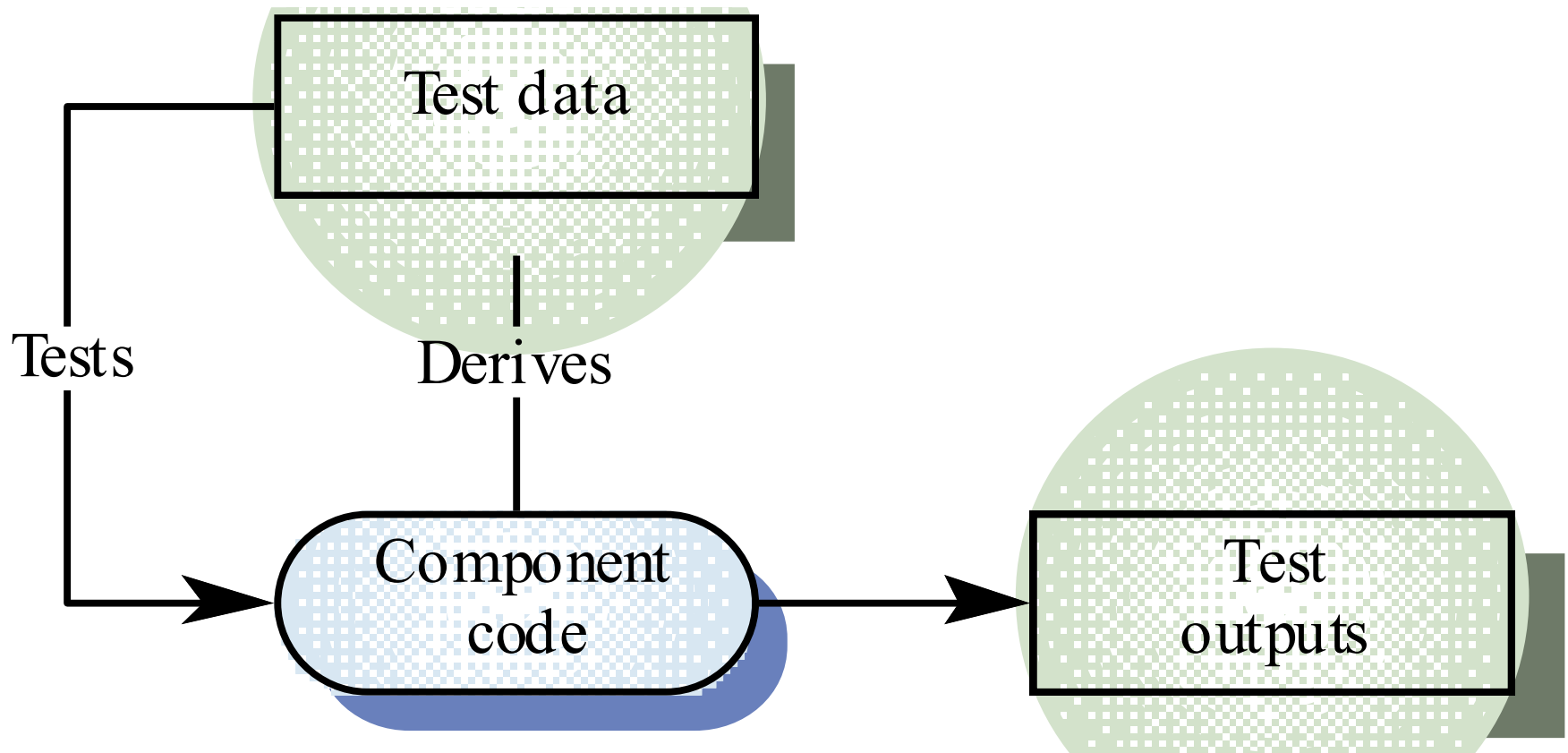
Array	Element
Single value	In sequence
Single value	Not in sequence
More than 1 value	First element in sequence
More than 1 value	Last element in sequence
More than 1 value	Middle element in sequence
More than 1 value	Not in sequence

Input sequence (T)	Key (Key)	Output (Found, L)
17	17	true, 1
17	0	false, ??
17, 29, 21, 23	17	true, 1
41, 18, 9, 31, 30, 16, 45	45	true, 7
17, 18, 21, 23, 29, 41, 38	23	true, 4
21, 23, 29, 33, 38	25	false, ??

Structural testing

- Sometimes called *white-box* testing
- Test cases are derived from the *program structure*
- Knowledge of the program is used to identify additional test cases
- Objective is to exercise all program statements (not all path combinations)

White-box testing



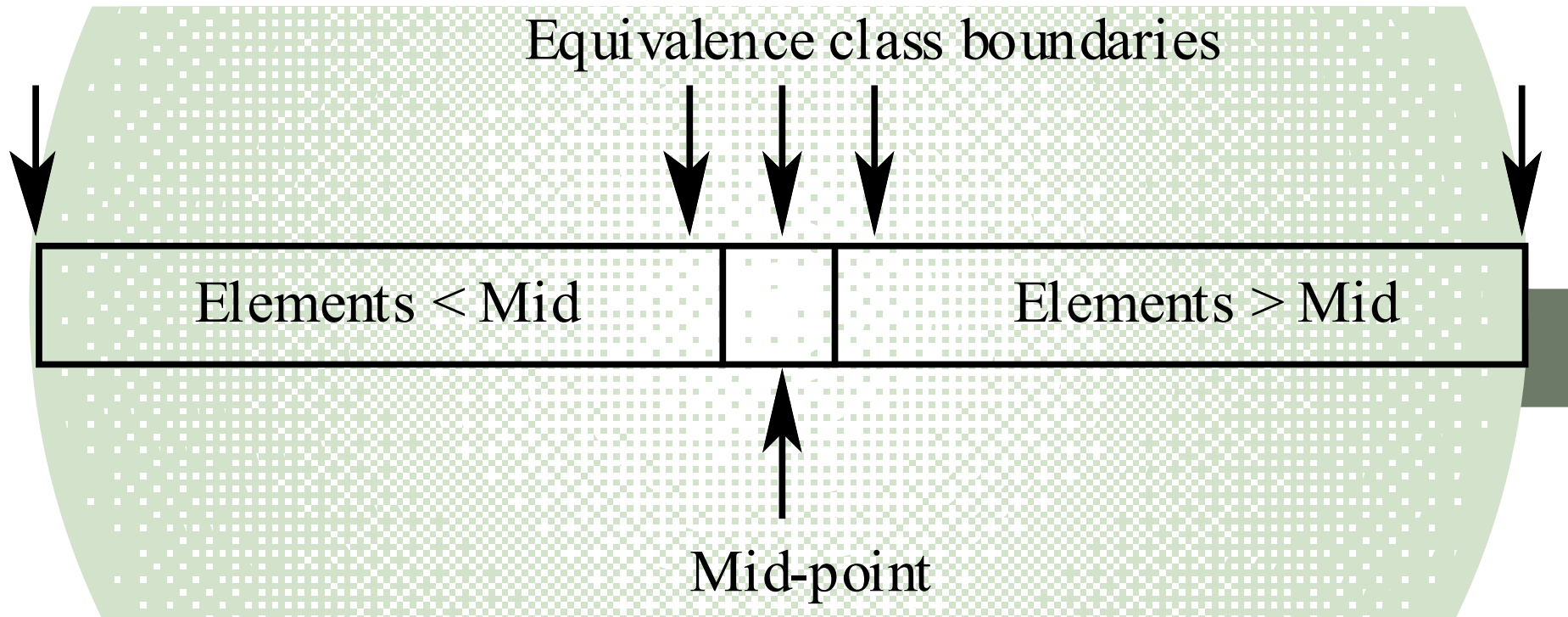
```
class BinSearch {
```

```
// This is an encapsulation of a binary search function that takes an array of  
// ordered objects and a key and returns an object with 2 attributes namely  
// index - the value of the array index  
// found - a boolean indicating whether or not the key is in the array  
// An object is returned because it is not possible in Java to pass basic types by  
// reference to a function and so return two values  
// the key is -1 if the element is not found
```

```
    public static void search ( int key, int [] elemArray, Result r )  
    {  
        int bottom = 0 ;  
        int top = elemArray.length - 1 ;  
        int mid ;  
        r.found = false ; r.index = -1 ;  
        while ( bottom <= top )  
        {  
            mid = (top + bottom) / 2 ;  
            if (elemArray [mid] == key)  
            {  
                r.index = mid ;  
                r.found = true ;  
                return ;  
            } // if part  
            else  
            {  
                if (elemArray [mid] < key)  
                    bottom = mid + 1 ;  
                else  
                    top = mid - 1 ;  
            }  
        } //while loop  
    } // search  
} //BinSearch
```

Binary search (Java)

Binary search equiv. partitions



Binary search - equiv. partitions

- Pre-conditions satisfied, key element in array
- Pre-conditions satisfied, key element not in array
- Pre-conditions unsatisfied, key element in array
- Pre-conditions unsatisfied, key element not in array
- Input array has a single value
- Input array has an even number of values
- Input array has an odd number of values

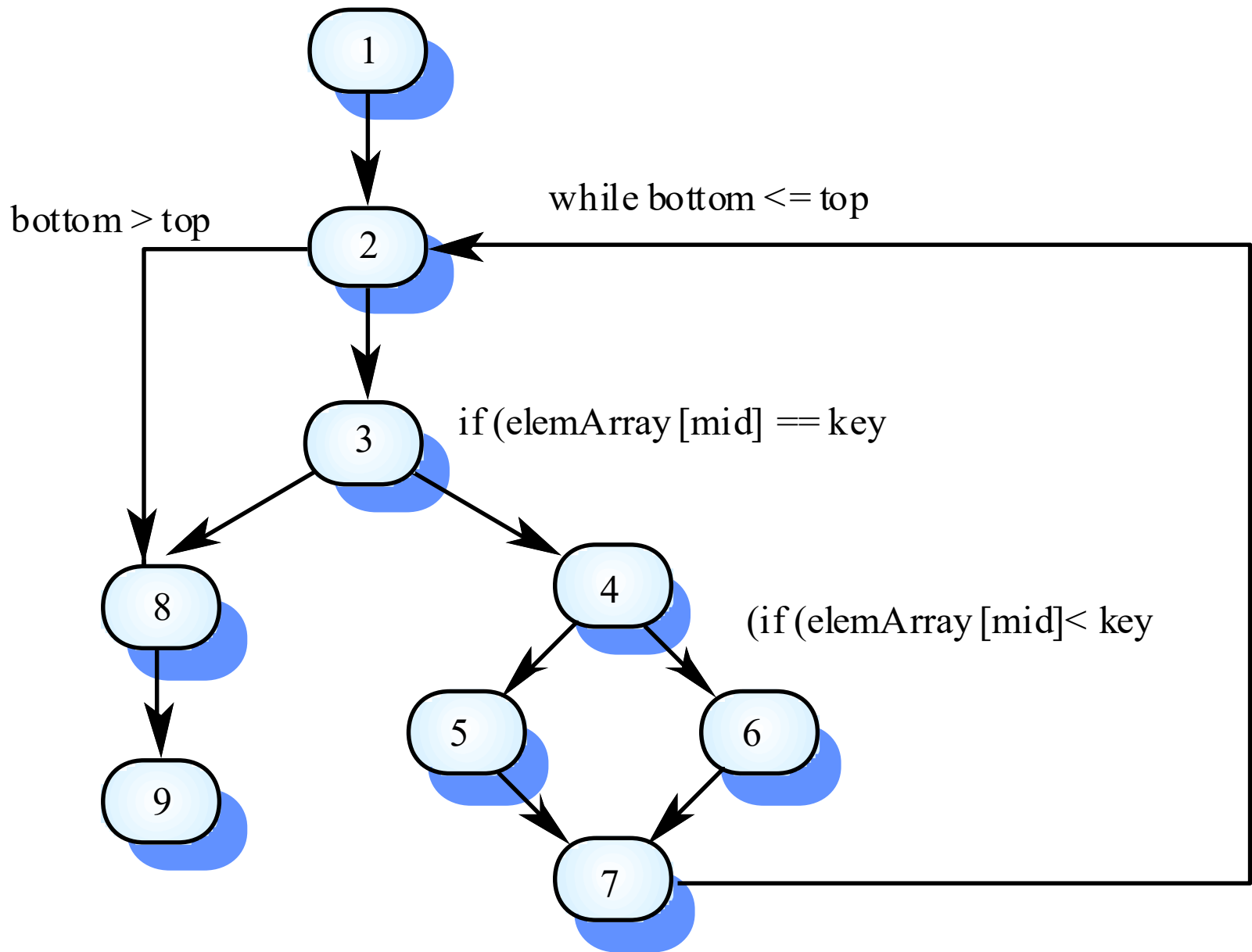
Binary search

Example test cases

Input array (T)	Key (Key)	Output (Found, L)
17	17	true, 1
17	0	false, ??
17, 21, 23, 29	17	true, 1
9, 16, 18, 30, 31, 41, 45	45	true, 7
17, 18, 21, 23, 29, 38, 41	23	true, 4
17, 18, 21, 23, 29, 33, 38	21	true, 3
12, 18, 21, 23, 32	23	true, 4
21, 23, 29, 33, 38	25	false, ??

Path testing

- The objective of path testing is to ensure that the set of test cases is such that each path through the program (and thus each statement) is executed at least once
- The number of paths through a program is usually proportional to its size. As modules are integrated into systems, it becomes unfeasible to use path testing techniques, that are therefore mostly used at unit or module testing stages
- The starting point for path testing is a program flow graph that shows nodes representing program decisions and arcs representing the flow of control



Binary search flow graph

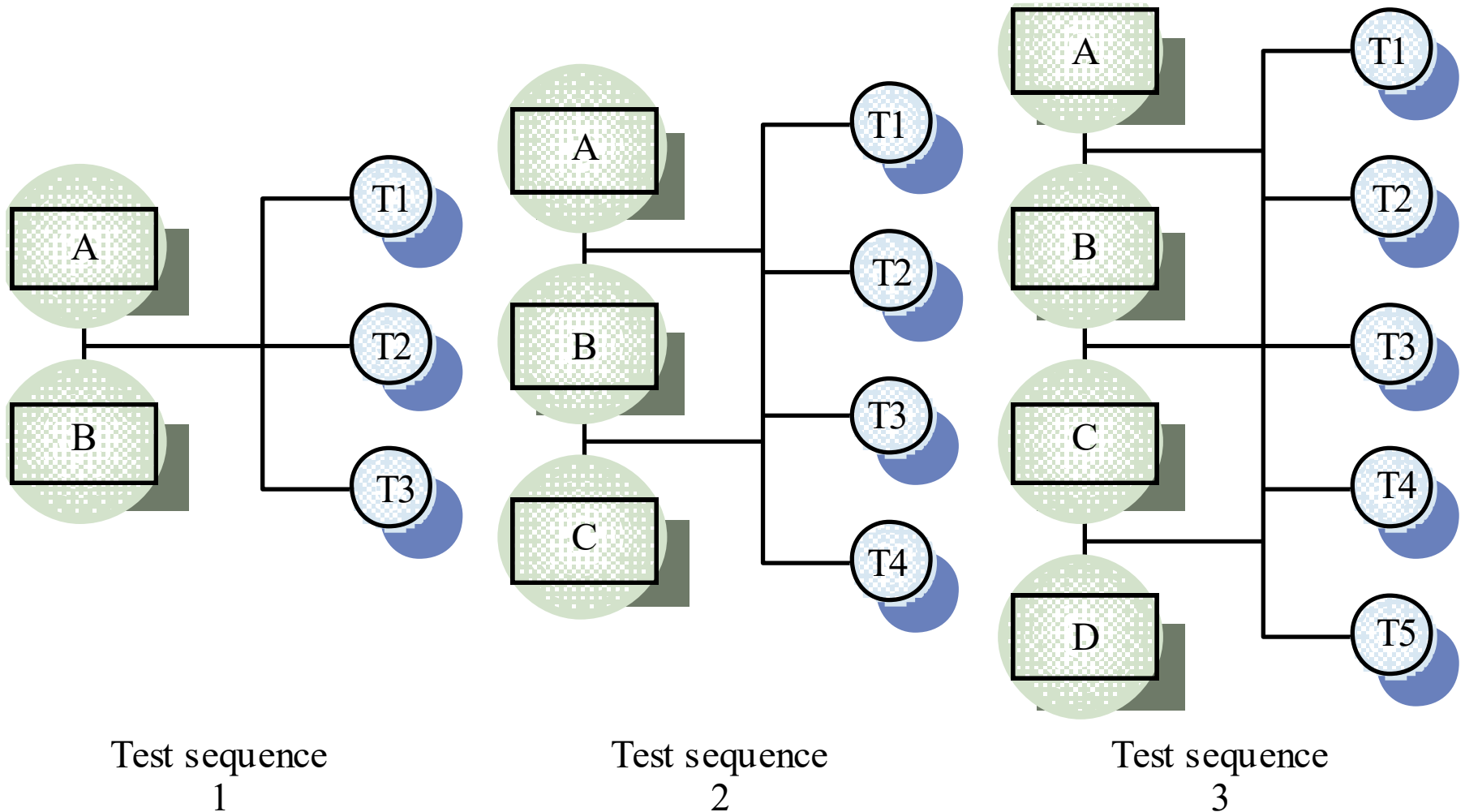
Independent paths

- 1, 2, 3, 8, 9
- 1, 2, 3, 4, 6, 7, 2
- 1, 2, 3, 4, 5, 7, 2
- 1, 2, 3, 4, 6, 7, 2, 8, 9
- Test cases should be derived so that all of these paths are executed
- *Dynamic program analysers or profilers* may be used to check that paths have been executed

Integration testing

- Tests complete systems or subsystems composed of integrated components
- Integration testing should be black-box testing with test cases derived from the specification
- Main difficulty is localising defects
- Incremental integration testing reduces this problem

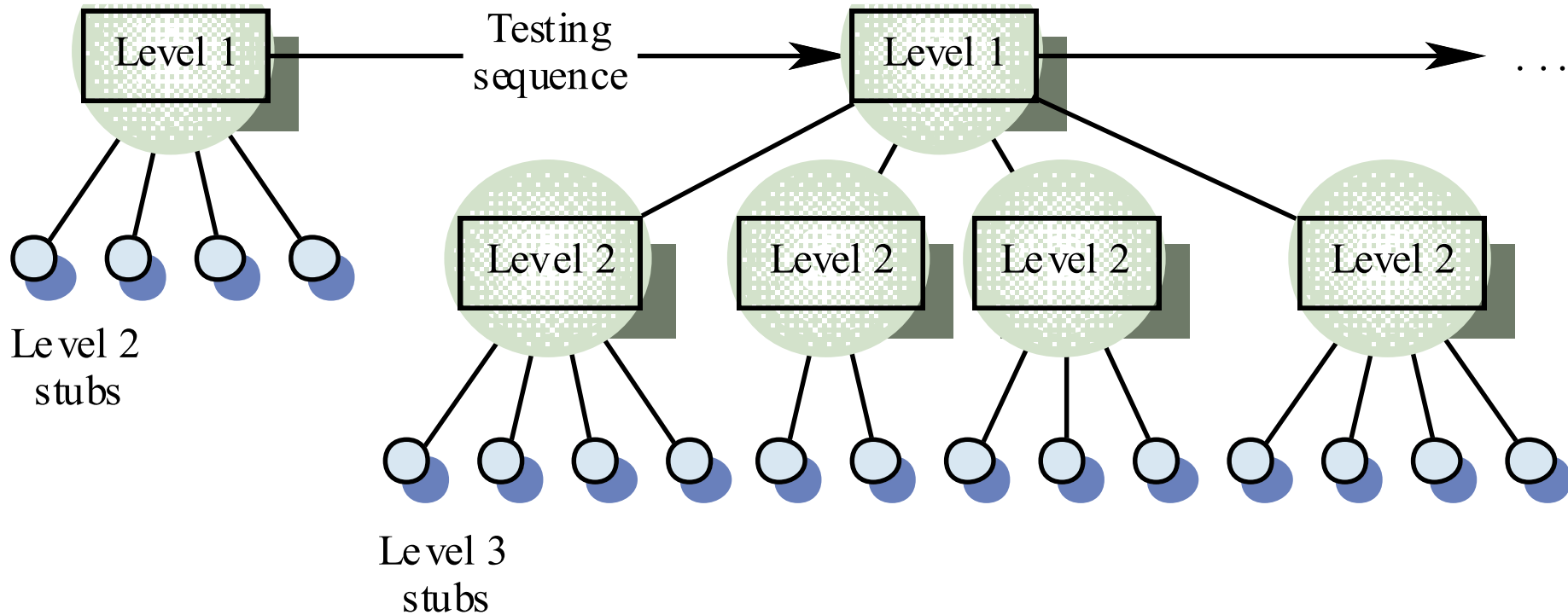
Incremental integration testing



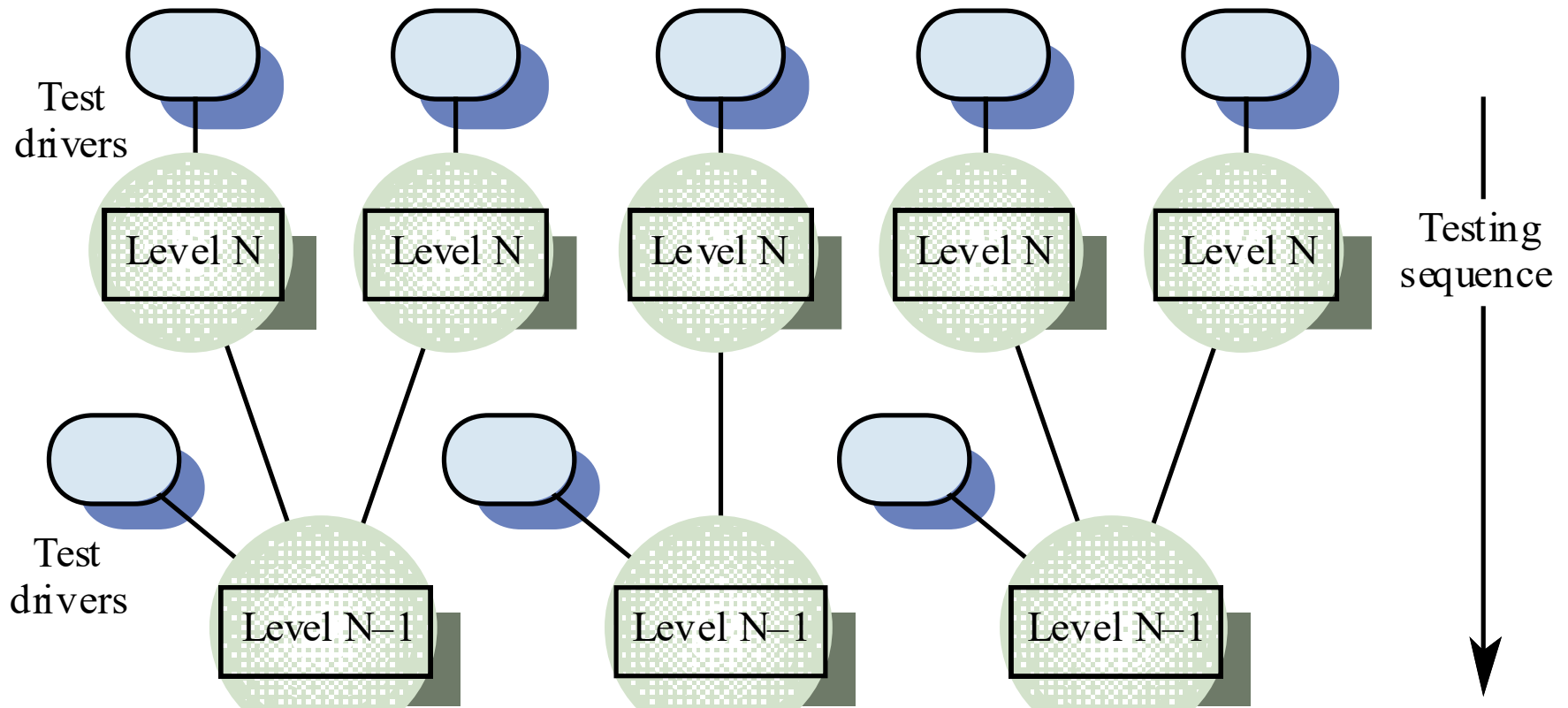
Approaches to integration testing

- Top-down testing
 - Start with high-level system and integrate from the top-down replacing individual components by stubs where appropriate
- Bottom-up testing
 - Integrate individual components in levels until the complete system is created
- In practice, most integration involves a combination of these strategies

Top-down testing



Bottom-up testing



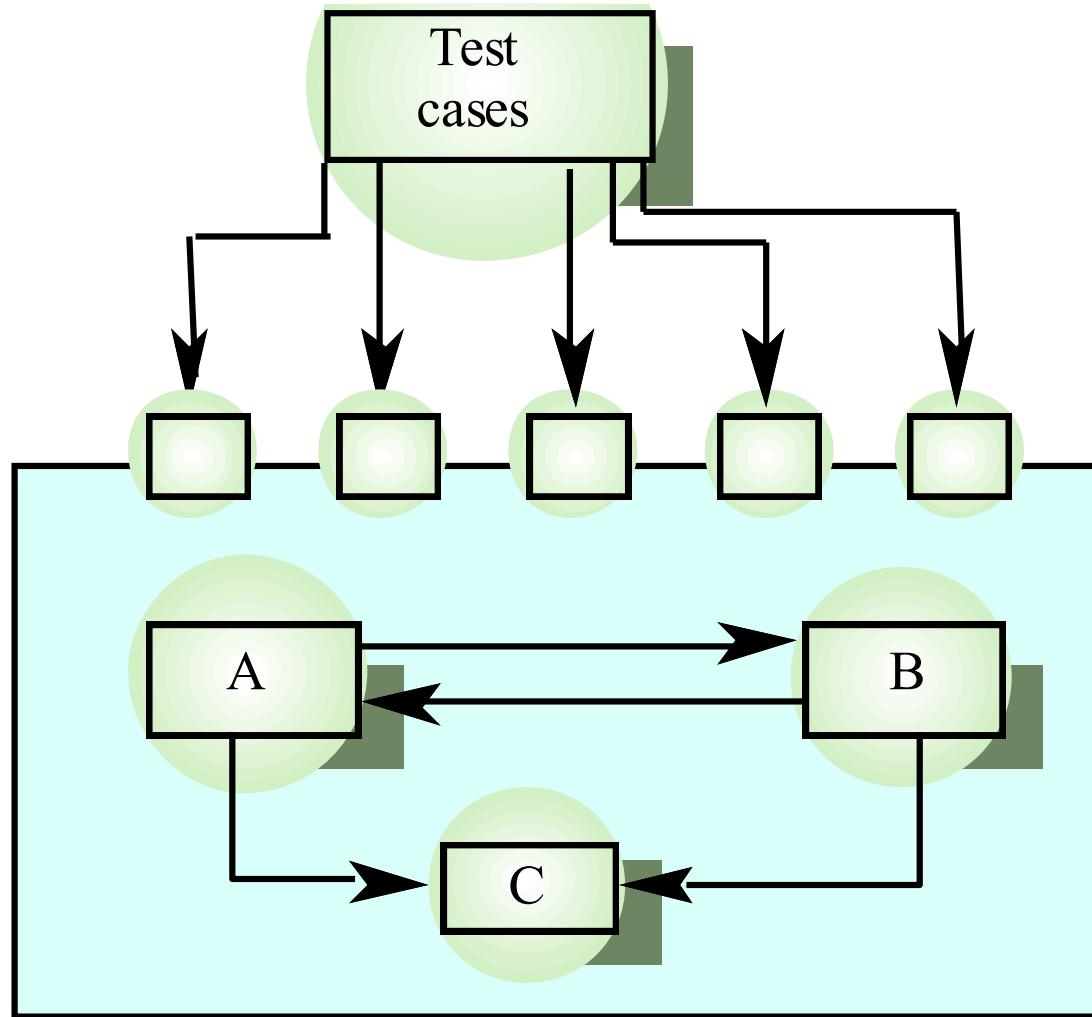
Testing approaches

- Architectural validation
 - Top-down integration testing is better at discovering defects in the system architecture
- System demonstration
 - Top-down integration testing allows a limited demonstration at an early stage in the development
- Test implementation
 - Often easier with bottom-up integration testing
- Test observation
 - Problems with both approaches. Extra code may be required to observe tests

Interface testing

- Takes place when modules or sub-systems are integrated to create larger systems
- Objectives are to detect faults due to interface errors or invalid assumptions about interfaces
- Particularly important for object-oriented development as objects are defined by their interfaces

Interface testing



Interfaces types

- Parameter interfaces
 - Data passed from one procedure to another
- Shared memory interfaces
 - Block of memory is shared between procedures
- Procedural interfaces
 - Sub-system encapsulates a set of procedures to be called by other sub-systems
- Message passing interfaces
 - Sub-systems request services from other sub-systems

Interface errors

- Interface misuse

A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order, of wrong type or in the wrong number

- Interface misunderstanding

A calling component embeds assumptions about the behaviour of the called component which are incorrect

- Timing errors

The called and the calling component operate at different speeds and out-of-date information is accessed

Interface testing guidelines

- Design tests so that parameters to a called procedure are at the extreme ends of their ranges
- Always test pointer parameters with null pointers
- Design tests which cause the component to fail
- Use stress testing in message passing systems
- In shared memory systems, vary the order in which components are activated

Stress testing

- Exercises the system beyond its maximum design load. Stressing the system often causes defects to come to light.
- Stressing the system tests failure behaviour
 - systems should not fail catastrophically
 - the failure should not cause unacceptable loss of service or data
- Particularly relevant to distributed systems which can exhibit severe degradation as a network becomes overloaded

Object-oriented testing

- The components to be tested are object classes that are instantiated as objects
- Larger grain than individual functions so approaches to white-box testing have to be extended
- No obvious 'top' to the system for top-down integration and testing

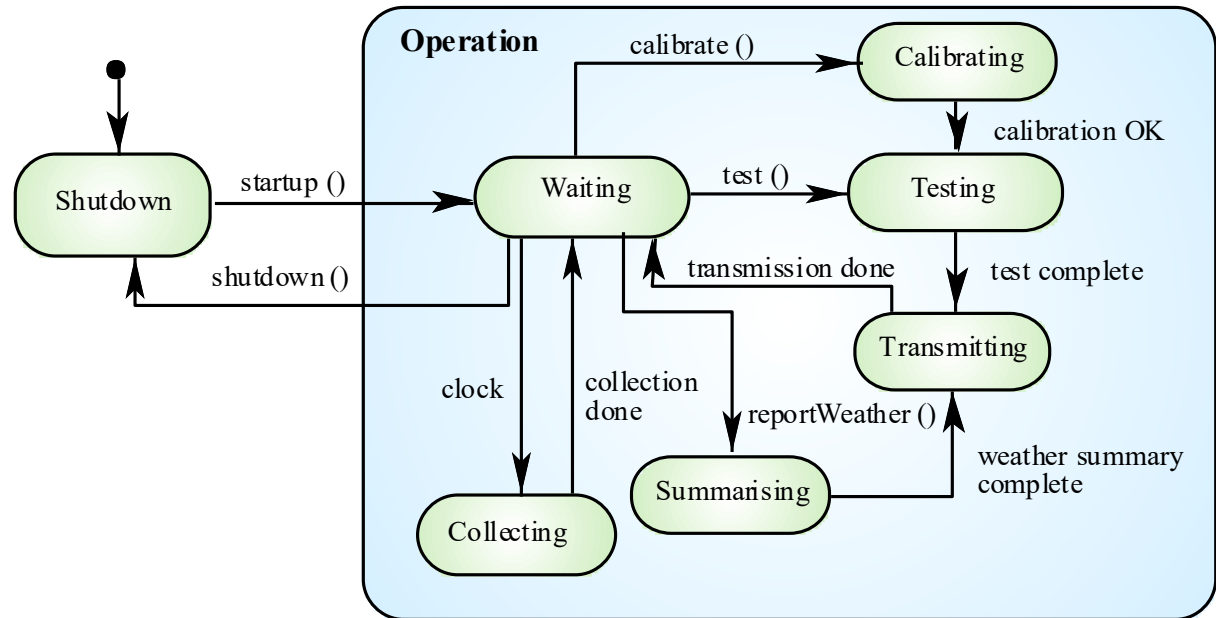
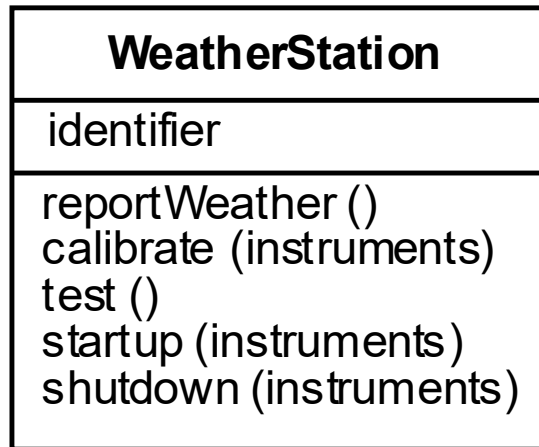
Testing levels

- Testing methods associated with objects
- Testing object classes
- Testing clusters of cooperating objects
- Testing the complete OO system

Object class testing

- Complete test coverage of a class involves
 - Testing all operations associated with an object
 - Setting and interrogating all object attributes
 - Exercising the object in all possible states
- Inheritance makes it more difficult to design object class tests as the information to be tested is not localised

Weather station class and state diagram



- Test cases are needed for all operations
- Use a state model to identify state transitions for testing
- Examples of testing sequences:
 - Shutdown → Waiting → Shutdown
 - Waiting → Calibrating → Testing → Transmitting → Waiting
 - Waiting → Collecting → Waiting → Summarising → Transmitting → Waiting

Objects integration

- Levels of integration are less distinct in object-oriented systems
- *Cluster testing* is concerned with integrating and testing clusters of cooperating objects
- Identify clusters using knowledge of the operation of objects and the system features that are implemented by these clusters

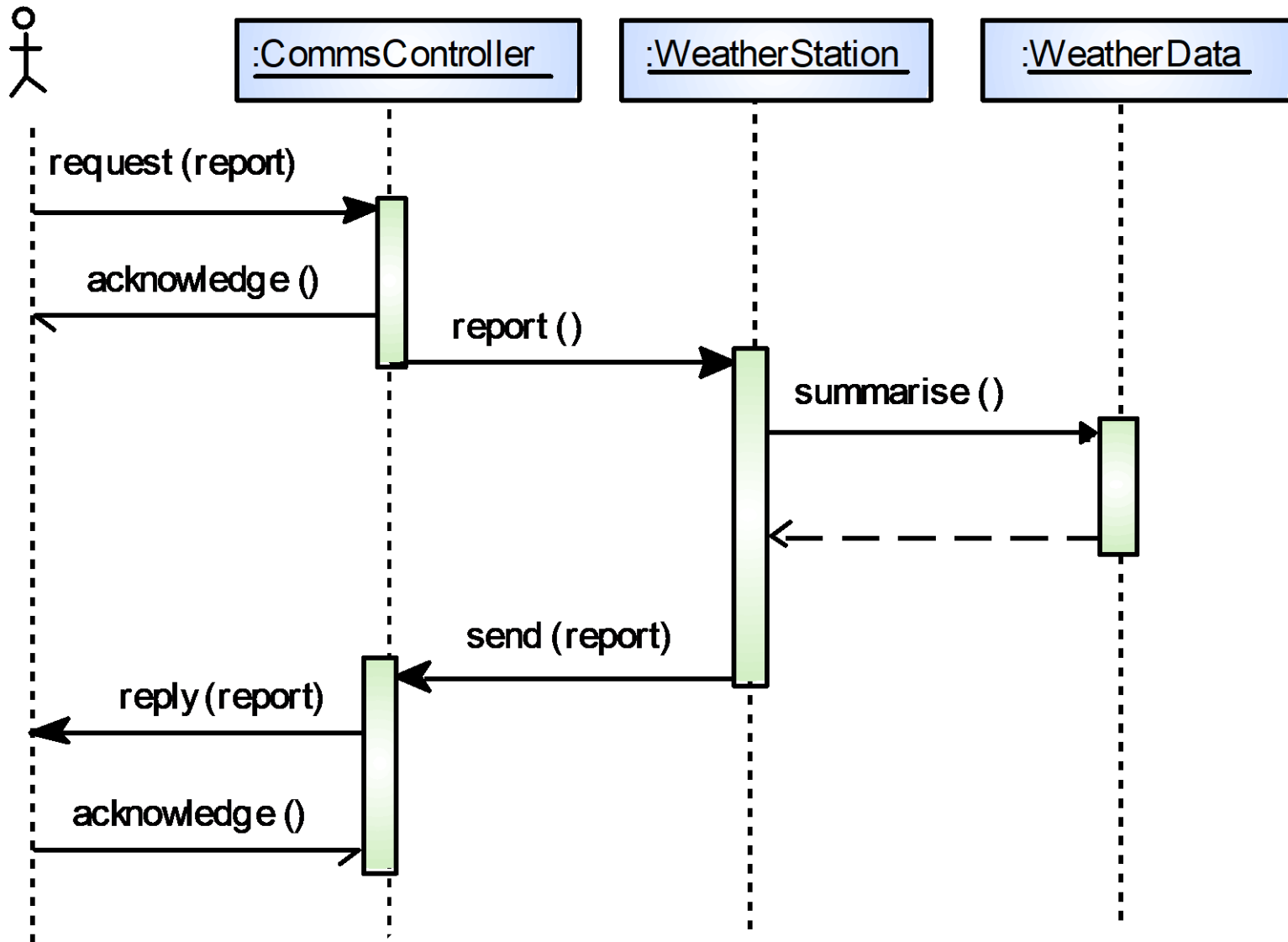
Approaches to cluster testing

- Use-case or scenario testing
 - Testing is based on a user interactions with the system
 - Has the advantage that it tests system features as experienced by users
- Thread testing
 - Tests the systems response to events as processing threads through the system
- Object interaction testing
 - Tests sequences of object interactions that stop when an object operation does not call on services from another object

Scenario-based testing

- Identify scenarios from use-cases and supplement these with interaction diagrams that show the objects involved in the scenario (i.e. sequence diagrams)
- Consider the scenario in the weather station system where a report is generated

Collect weather data



Testing workbenches

- Testing is an expensive process phase
- Testing workbenches provide a range of tools to reduce the time required and total testing costs
- Most testing workbenches are open systems because testing needs are organisation-specific

A testing workbench

